

CAD-ANALYZER

Fichas de Extração — PILARES

■ Esta ficha responde: dado um texto/polígono no DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_PILAR · padrões · exemplos
2	Associação texto→polígono	3 raios · scores · algoritmo + diagrama
3	Seção Transversal	comprimento·largura·cambotado + diagrama
4	Layers	NOMENCLATURA·Painéis·aliases + diagrama
5	Schema JSON Completo	FichaFase3Pilar · todos os campos
6	Confidence e Fallbacks	fórmula + diagrama visual · cadeia de 5 níveis
7	Matriz de Decisão	todos os casos ambíguos
8	Exemplos Reais ALIMONTI	P17 (1.0) · P5 (0.394) · PC-1 cambotado
9	Pipeline Completo	fluxo E2E + diagrama · checklist

1

Identificação — RE_PILAR

```
RE_PILAR = re.compile(
    r'^(PC?\.\.?-\?\d+([A-Z]|\.\.\?\d+)?)'
    r'|P-\d+[A-Z]?'
    r'|PILAR[-_\s]*\d+)',
    re.IGNORECASE
)
```

Texto DXF	Casou?	Campo extraído
P17	✓ SIM	codigo = "P17"
P17A	✓ SIM	codigo = "P17A" (suífixo letra)
PC-1	✓ SIM	codigo = "PC-1" (pilar de canto)
P.17	✓ SIM	codigo = "P.17"
P-8	✓ SIM	codigo = "P-8"
PILAR 3	✓ SIM	codigo = "PILAR 3"
V5	✗ NÃO	– (é viga, não pilar)
P	✗ NÃO	– (sem número)

```
for e in msp:
    if e.dxf.type() not in ('TEXT', 'MTEXT'): continue
    txt = e.dxf.text.strip() if e.dxf.type() == 'TEXT' else e.plain_text().strip()
    x, y = float(e.dxf.insert.x), float(e.dxf.insert.y)
    layer = e.dxf.layer
    if RE_PILAR.match(txt):
        pilares_txt.append({'text': txt, 'x': x, 'y': y, 'layer': layer})
```

■ Um texto "P17" sozinho NÃO é pilar. Precisa de LWPOLYLINE fechada próxima.

Lógica 3 Raios – TextAssociator (Pilares)

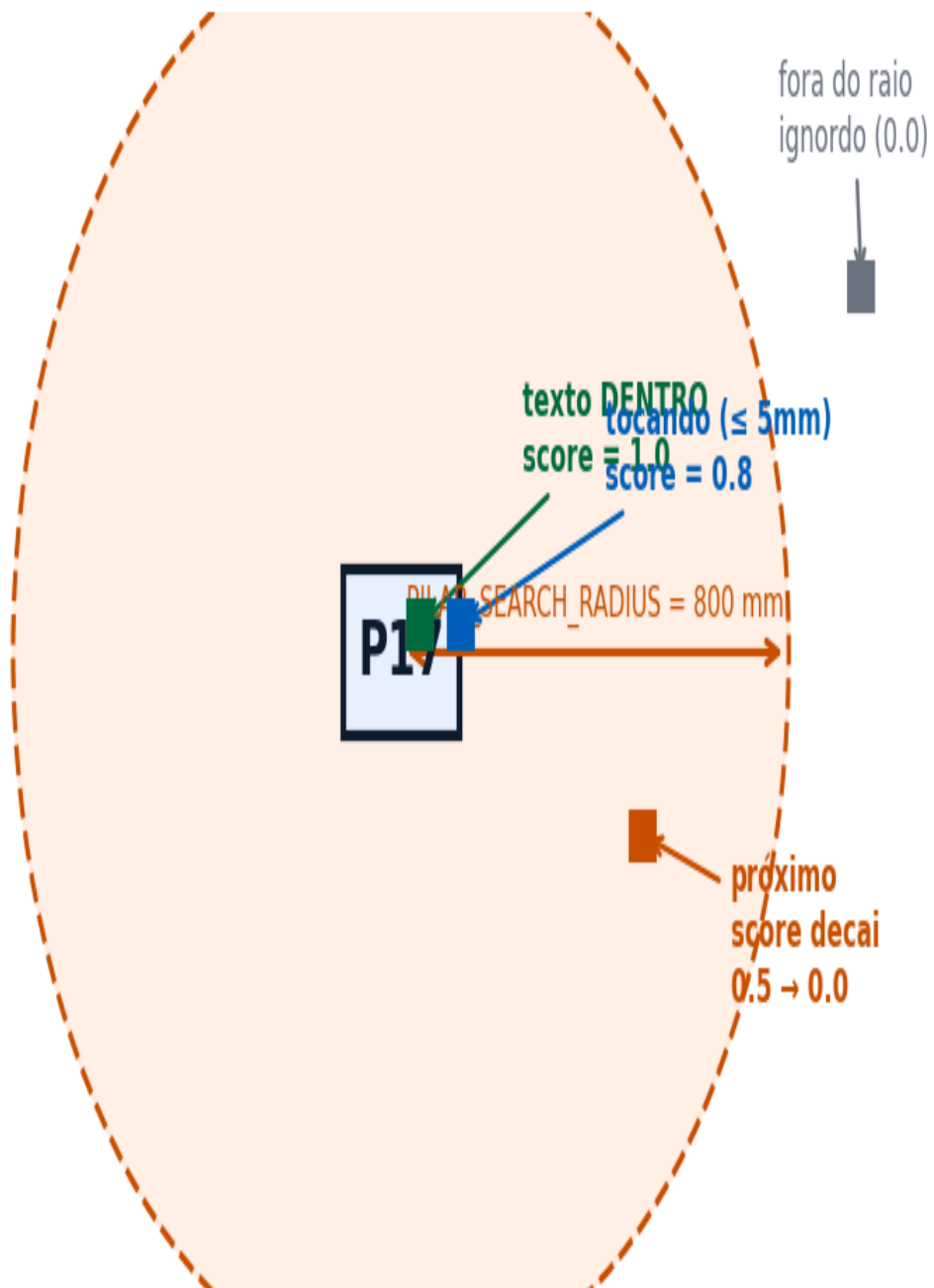


Figura 1 — TextAssociator: 3 zonas de associação texto→polígono

```
from shapely.geometry import Point, Polygon
PILAR_SEARCH_RADIUS = 800.0 # mm
TOUCH_DIST = 5.0 # mm

def score_texto_poligono(txt_pt, poly_pts):
    poly = Polygon(poly_pts); pt = Point(txt_pt)
    dist = poly.exterior.distance(pt)
    if poly.contains(pt): return 1.0
    elif dist <= TOUCH_DIST: return 0.8
    elif dist <= PILAR_SEARCH_RADIUS:
        return max(0.0, 0.5*(1.0 - dist/PILAR_SEARCH_RADIUS))
    return 0.0
```

Seção Transversal do Pilar — Tipos

Pilar Retangular

Pilar Cambotado (bulge > 0.3)

comprimento $\geq$ largura (sempre)



pilar_especial = True
tipo = "CAMBOTADO"

bulge > 0.3
neste vértice



Figura 2 — Pilar retangular (comprimento $\geq$ largura) e cambotado (bulge > 0.3)

```
# Formatos aceitos:
RE_DIM = re.compile(r'(\d{1,3})\s*[xX*\/]\s*(\d{1,3})')
RE_DIM_BH = re.compile(r'b\s*=\s*(\d{1,3}).*?h\s*=\s*(\d{1,3})', re.I)

# Normalização: comprimento >= largura
comp = max(a, b); larg = min(a, b) # cm

# Cambotado (arco na LWPOLYLINE):
for pt in lwpoly.get_points('xyb'): # x, y, bulge
    if abs(pt[2]) > 0.3:
        pilar_especial = True; tipo = 'CAMBOTADO'
```

Caso	Condição	Ação
Pilar retangular normal	bulge == 0	comprimento = max(a,b), largura = min(a,b)
Pilar cambotado	bulge > 0.3	pilar_especial=True, tipo="CAMBOTADO", usar bbox
Sem dimensão no raio	nenhum texto NNxMM em 600mm	comp=0, larg=0, confidence=0.30
Coordenadas UTM	x ou y > 50000	Ignorar — georreferenciamento, não fôrmas

Layers do Pilar – O que cada layer contém

Layer: SARrafo, SARR_2.2x7, CHAPA, etc → componentes de fôrma → extraídos separadamente

Layer: Painéis (pode vir "Pain?is" CP1252) → `WPPOLINE` → `True` → contorno do pilar → `outline_segs[]`

Layer: NOMENCLATURA → `TEXT/MTEXT` → "P17" → texto ID do pilar

Layer: COTA → `TEXT/MTEXT` → "20x50" → comprimento=50cm, largura=20cm

`normalize_layer("Painéis") == normalize_layer("Pain?is") == "PAINEIS"`

Figura 3 — Estrutura de layers do pilar: o que cada um contém

Canônico	Aliases reais no DXF	Uso
ELEMENT_LABEL	NOMENCLATURA, texto, "00 - FELIPE", EST-PILAR-TEXT	Textos ID (P17, V101, L5)

Canônico	Aliases reais no DXF	Uso
PANEL_GEOMETRY	Painéis, PAINEIS, "Pain?is", PAINEL	LWPOLYLINE fechada — contorno
WOOD_BATTEN	SARRAFO, "SARRAFO DE PRESSAO"	Sarrafos de madeira
BATTEN_2x7	SARR_2.2x7, "Sarr 2.2x7"	Sarrafo 2,2×7cm (mais comum)
BATTEN_7x7	SARR_7x7, SARR_7x10	Cantos de pilar
ANCHOR_BAR_PL	"BARRA ANCORAGEM"	Barras (≠ LV: "BARRA DE ANCORAGEM")
ELEVATION_MARK	NIVEL, "Nível", "N?vel"	Cota Z — TEXT/MTEXT
SECTION_TEXT	"Texto Seção", "Texto de Titulo"	Textos "20x50", "b=20 h=50"
TQS_COLUMN	S-COLS, "1", "2", "3"	Pilar família TQS

```
import unicodedata
def normalize_layer(name):
    nfkd = unicodedata.normalize('NFKD', str(name))
    return nfkd.encode('ascii', 'ignore').decode().upper().strip()

# normalize_layer('Painéis') == normalize_layer('Pain?is') == 'PAINEIS'
```



```

{
  "codigo":          "P17",          # str - ID
  "pavimento":       "1_PAVIMENTO",  # str - nome do arquivo DXF
  "obra_nome":       "ALIMONTI",      # str

  "comprimento":     50.0,            # float cm - dimensão maior
  "largura":         20.0,            # float cm - dimensão menor
  "pilar_especial":  false,           # bool
  "tipo_pilar_especial": "",          # "CAMBOTADO" | ""

  "outline_segs": [                  # LWPOLYLINE em mm
    {"x": 15000.0, "y": 10000.0},
    {"x": 15200.0, "y": 10000.0},
    {"x": 15200.0, "y": 10500.0},
    {"x": 15000.0, "y": 10500.0}
  ],

  "nivel":           2.80,            # float m - cota Z
  "armadura": {"tipo": "longitudinal", "diametro": 12.5, "espacamento": 15},
  "vigas_ligadas":  ["V101", "V102"], # inferido por proximidade

  "confidence":      0.85,
  "revisado":        false
}

```

Campo	Tipo	Origem DXF
codigo	str	TEXT/MTEXT layer NOMENCLATURA → RE_PILAR
comprimento	float	RE_DIM/RE_DIM_BH mais próximo $\leq 600\text{mm}$ → $\max(a,b)$
largura	float	idem → $\min(a,b)$
outline_segs	list	LWPOLYLINE closed=True layer Painéis
pilar_especial	bool	bulge > 0.3 em qualquer vértice
nivel	float	TEXT layer NIVEL → RE_NIVEL → valor em metros
confidence	float	calcular_confidence(raio, dim, id, contorno)

Fórmula de Confidence – Pilares

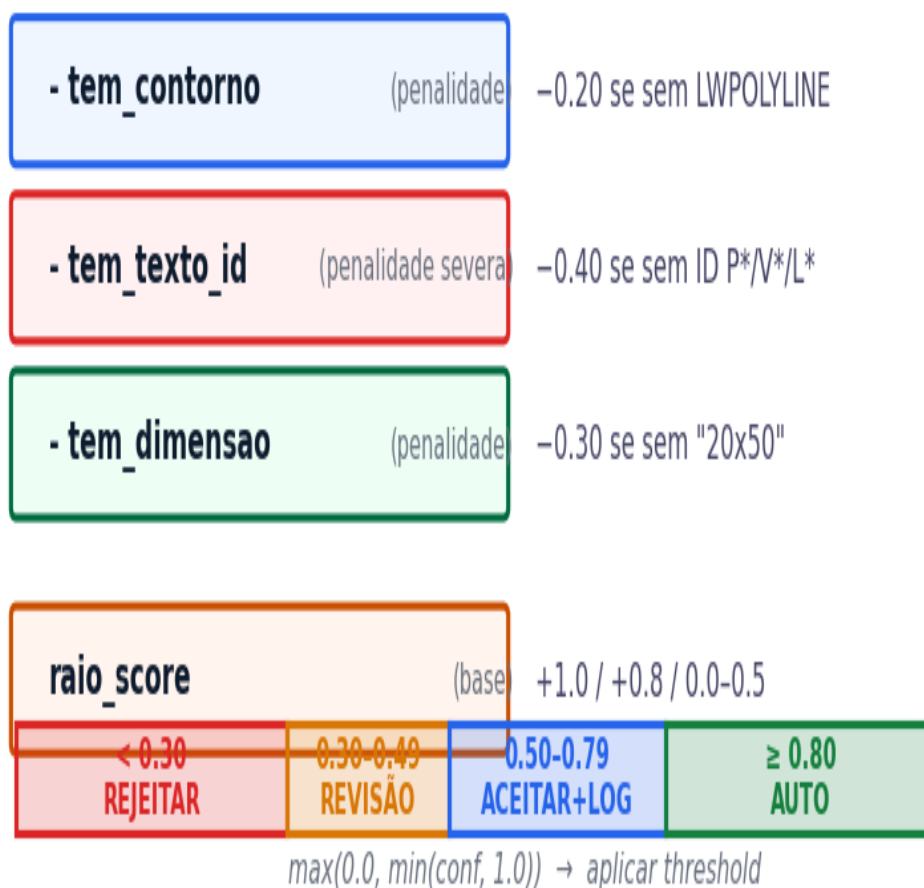


Figura 4 — Composição do score de confidence e thresholds

```
def calcular_confidence(raio_score, tem_dimensao, tem_texto_id, tem_contorno):
    conf = raio_score
    if not tem_dimensao: conf -= 0.30
    if not tem_texto_id: conf -= 0.40 # penalidade severa
    if not tem_contorno: conf -= 0.20
    return max(0.0, min(conf, 1.0))
```

Cadeia de Fallbacks

Nível	Condição	Confidence
1	Texto em NOMENCLATURA → LWPOLYLINE em Painéis	raio_score

Nível	Condição	Confidence
2	Texto em TEXTO_GERAL → mesma LWPOLYLINE	raio_score – 0.05
3	Texto em qualquer layer → LWPOLYLINE qualquer	raio_score – 0.15
4	Texto RE_PILAR sem LWPOLYLINE próxima	raio_score – 0.40
5	Nenhum texto RE_PILAR	NÃO REGISTRAR

Situação	Ação	Confidence	Log
Texto dentro do polígono	Auto-assign	1.0	—
Texto tocando ($\text{dist} \leq 5\text{mm}$)	Auto-assign	0.8	—
Texto próximo ($\leq 800\text{mm}$)	Score decaimento	0.0–0.5	avisar se < 0.50
Texto fora do raio ($> 800\text{mm}$)	Ignorar	0.0	—
2 textos competindo	Vence score maior	vencedor	log ambos
Empate exato de score	Revisão humana	score	log "EMPATE"
ID sem dimensão próxima	Registrar sem dim	conf–0.30	"dim não encontrada"
Layer desconhecido	Processar mesmo assim	conf–0.10	"layer UNKNOWN"
Encoding "Pain?is"	normalize_layer()	sem penalidade	—

Exemplo A — P17 (texto dentro do polígono → confidence = 1.0)

```
# DXF:
TEXT layer='NOMENCLATURA' text='P17' insert=(15100,10250)
TEXT layer='NOMENCLATURA' text='20x50' insert=(15050,10350)
LWPOLYLINE layer='Paineis' closed=True
  vertices=[(15000,10000),(15200,10000),(15200,10500),(15000,10500)]

# Cálculo:
# texto P17 está DENTRO do polígono → raio_score = 1.0
# tem_dimensao=True, tem_contorno=True → penalidades = 0
# confidence = 1.0 → AUTO-ASSIGN

# JSON: {"codigo":"P17","comprimento":50.0,"largura":20.0,"confidence":1.0}
```

Exemplo B — P5 (texto distante → rejeitado)

```
# Texto P5 a 315mm do polígono mais próximo, sem dimensão próxima:
# raio_score = 0.5 * (1 - 315/800) = 0.304
# tem_dimensao=False → conf -= 0.30 → conf = 0.004
# 0.004 < 0.30 → REJEITAR — não gravar no banco
# Log: {"elemento_id":"P5","confidence":0.004,"motivo":"texto distante+sem dim"}
```

Exemplo C — PC-1 cambotado

```
# LWPOLYLINE com bulge = 0.414 em um vértice → cambotado
# {"codigo":"PC-1","pilar_especial":true,"tipo_pilar_especial":"CAMBOTADO",
#  "comprimento":40.0,"largura":40.0,"confidence":0.85}
```


Pipeline de Extração – Pilares



Figura 5 — Fluxo completo de extração de pilares (11 passos)